

COMP 3250

Data Analytics

ENSEMBLE MODELS

© DR. ABDULRAUF GIDADO
WITH EXAMPLES AND FIGURES FROM
INTRODUCTION TO MACHINE LEARNING WITH
PYTHON: A GUIDE FOR DATA SCIENTISTS
BY ANDREAS C. MÜLLER, SARAH GUIDO · 2016

Agenda



Random Forests



Gradient Boosted Decision Trees



Information about Midterm



Information about Project

© DR. ABDULRAUF GIDADO

WITH EXAMPLES AND FIGURES FROM INTRODUCTION TO
MACHINE LEARNING WITH PYTHON: A GUIDE FOR DATA
SCIENTISTS BY ANDREAS C. MÜLLER, SARAH GUIDO, 2016

Let's do a quick recap on decision trees



While we focused our discussion here on decision trees for classification, all that was said is similarly true for decision trees for regression, as implemented in `DecisionTreeRegressor`.

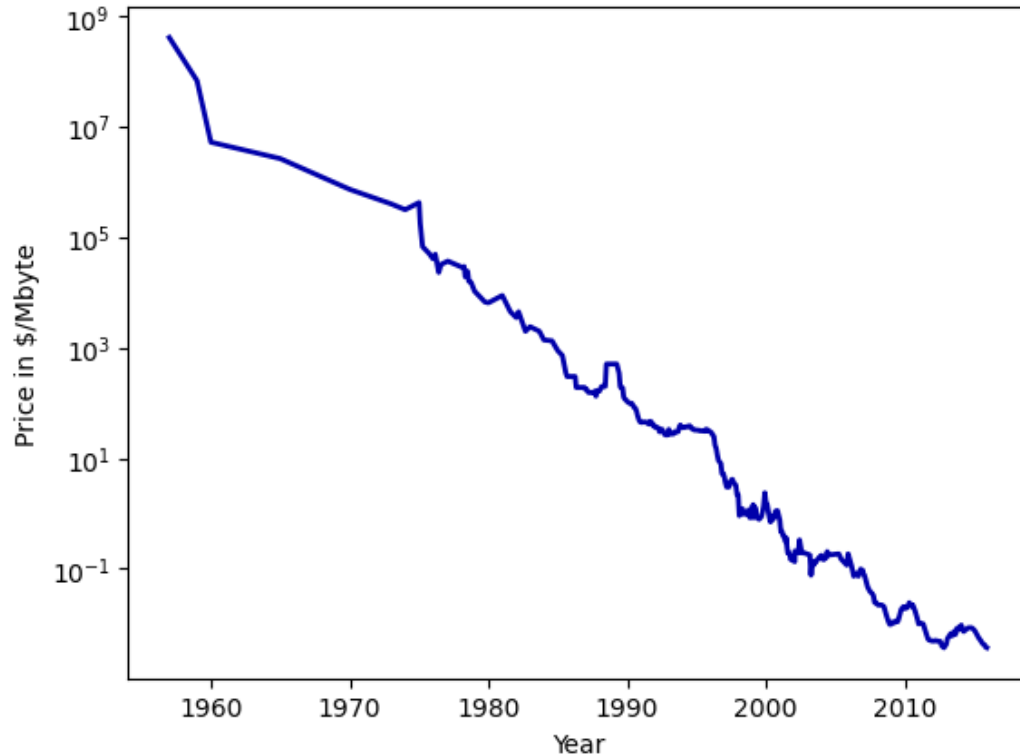


The usage and analysis of regression trees is very similar to that of classification trees.



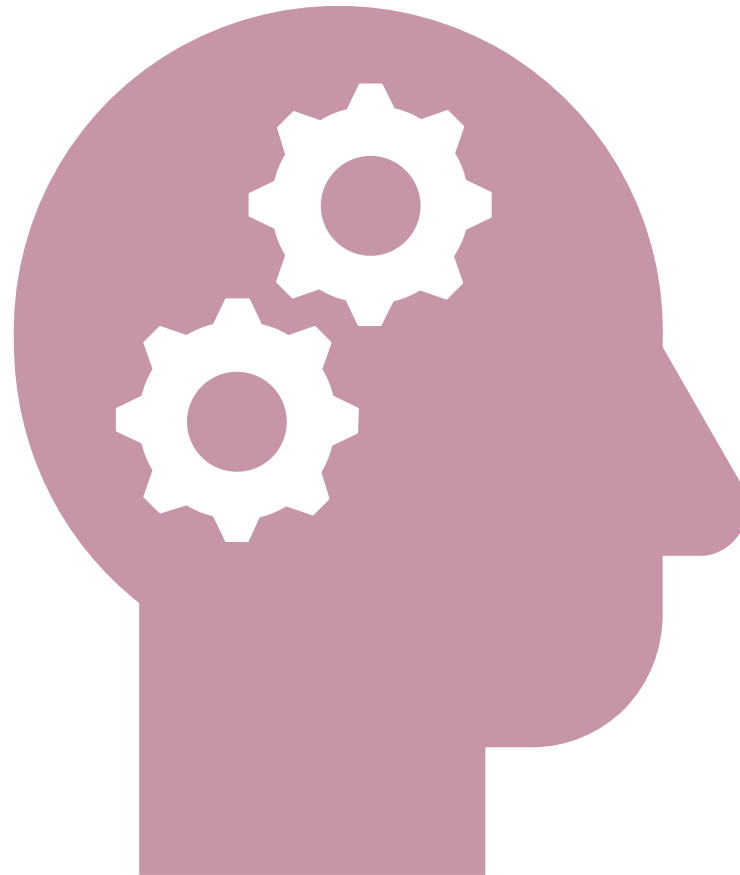
The `DecisionTreeRegressor` (and all other tree-based regression models) cannot extrapolate, or make predictions outside of the range of the training data.

To better understand DecisionTreeRegressor



Note the logarithmic scale of the y-axis. When plotting logarithmically, the relation seems to be quite linear and so should be relatively easy to predict, apart from some bumps.

- using a dataset of historical computer memory (RAM) prices:
- `import pandas as pd`
- `ram_prices = pd.read_csv("data/ram_price.csv")`
- `plt.semilogy(ram_prices.date, ram_prices.price)`
- `plt.xlabel("Year")`
- `plt.ylabel("Price in $/Mbyte")`



Now that we have visualized the dataset, let's get some training and prediction to confirm our earlier assertions about `DecisionTreeRegressor` and `DecisionTreeClassifier` in Slide 3 above!

© DR. ABDULRAUF GIDADO

WITH EXAMPLES AND FIGURES FROM INTRODUCTION
TO MACHINE LEARNING WITH PYTHON: A GUIDE FOR
DATA SCIENTISTS BY ANDREAS C. MÜLLER, SARAH
GUIDO · 2016



Great!
Let's get to
action

Note



We will compare two simple models: a `DecisionTreeRegressor` and `LinearRegression`



We rescale the prices using a logarithm, so that the relationship is relatively linear. This doesn't make a difference for the `DecisionTreeRegressor`, but it makes a big difference for `LinearRegression` (We will look at this more around week 9)

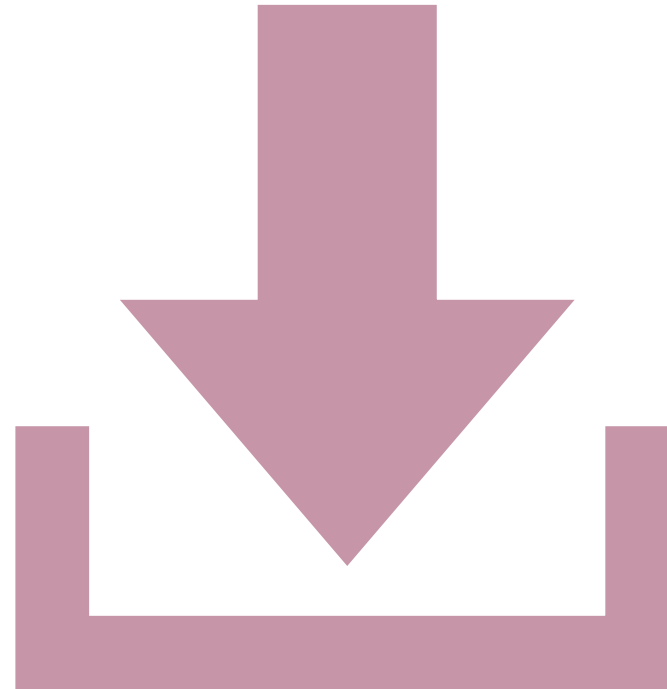


After training the models and making predictions, we apply the exponential map to undo the logarithm transform.

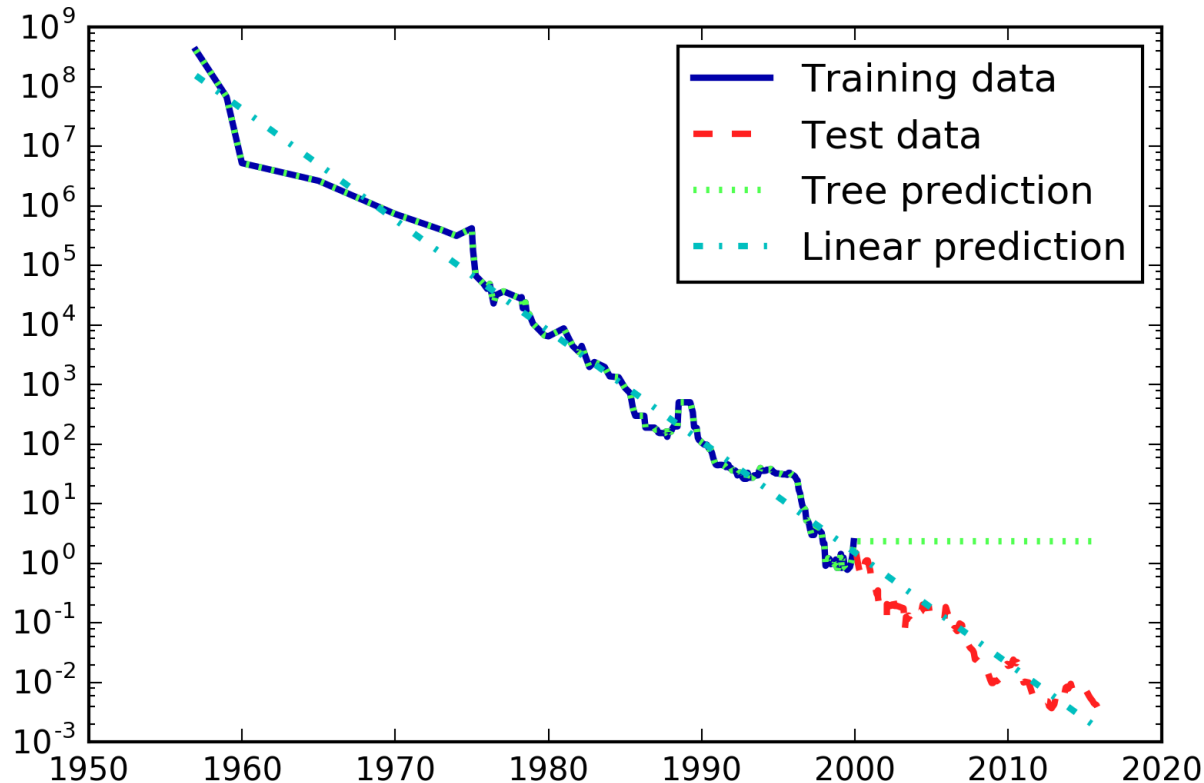


We make predictions on the whole dataset for visualization purposes here, but for a quantitative evaluation we would only consider the test dataset

Let's get into the ipynb file



This Figure compares the predictions of the decision tree and the linear regression model with the ground truth:



- `plt.semilogy(data_train.date, data_train.price, label="Training data")`
- `plt.semilogy(data_test.date, data_test.price, label="Test data")`
- `plt.semilogy(ram_prices.date, price_tree, label="Tree prediction")`
- `plt.semilogy(ram_prices.date, price_lr, label="Linear prediction")`
- `plt.legend()`

The tree model, on the other hand, makes perfect predictions on the training data; we did not restrict the complexity of the tree, so it learned the whole dataset by heart. However, once we leave the data range for which the model has data, the model simply keeps predicting the last known point. The tree has no ability to generate “new” responses, outside of what was seen in the training data. This shortcoming applies to all models based on trees.

Summary on Decision Trees: Strengths, weaknesses, and parameters

- The parameters that control model complexity in decision trees are the pre-pruning parameters that stop the building of the tree before it is fully developed
- Usually, picking one of the pre-pruning strategies—setting either `max_depth`, `max_leaf_nodes`, or `min_samples_leaf`—is sufficient to prevent overfitting.
- Advantages: Decision trees have two advantages over many of the algorithms we've discussed so far in this course
 1. The resulting model can easily be visualized and understood by nonexperts (at least for smaller trees)
 2. The algorithms are completely invariant to the scaling of the data. Why???

Why is decision tree algorithm completely invariant to the scaling of the data?



- As each feature is processed separately, and the possible splits of the data don't depend on scaling, no preprocessing like normalization or standardization of features is needed for decision tree algorithms.
- In particular, decision trees work well when you have features that are on completely different scales, or a mix of binary and continuous features.

Weakness of the Decision Tree Model

- The main downside of decision trees is that even with the use of pre-pruning, they tend to overfit and provide poor generalization performance.



Hmm...

HOW CAN WE RESOLVE THIS
DOWNSIDE OF DECISION
TREE?





The solution is the “Ensembles of Decision Trees” Then, what are Ensembles?

© DR. ABDULRAUF GIDADO

WITH EXAMPLES AND FIGURES FROM INTRODUCTION
TO MACHINE LEARNING WITH PYTHON: A GUIDE FOR
DATA SCIENTISTS BY ANDREAS C. MÜLLER, SARAH
GUIDO · 2016

We will look at the two most effective ensemble models of decision tree. They are effective wide range of datasets for classification and regression.



RANDOM FORESTS



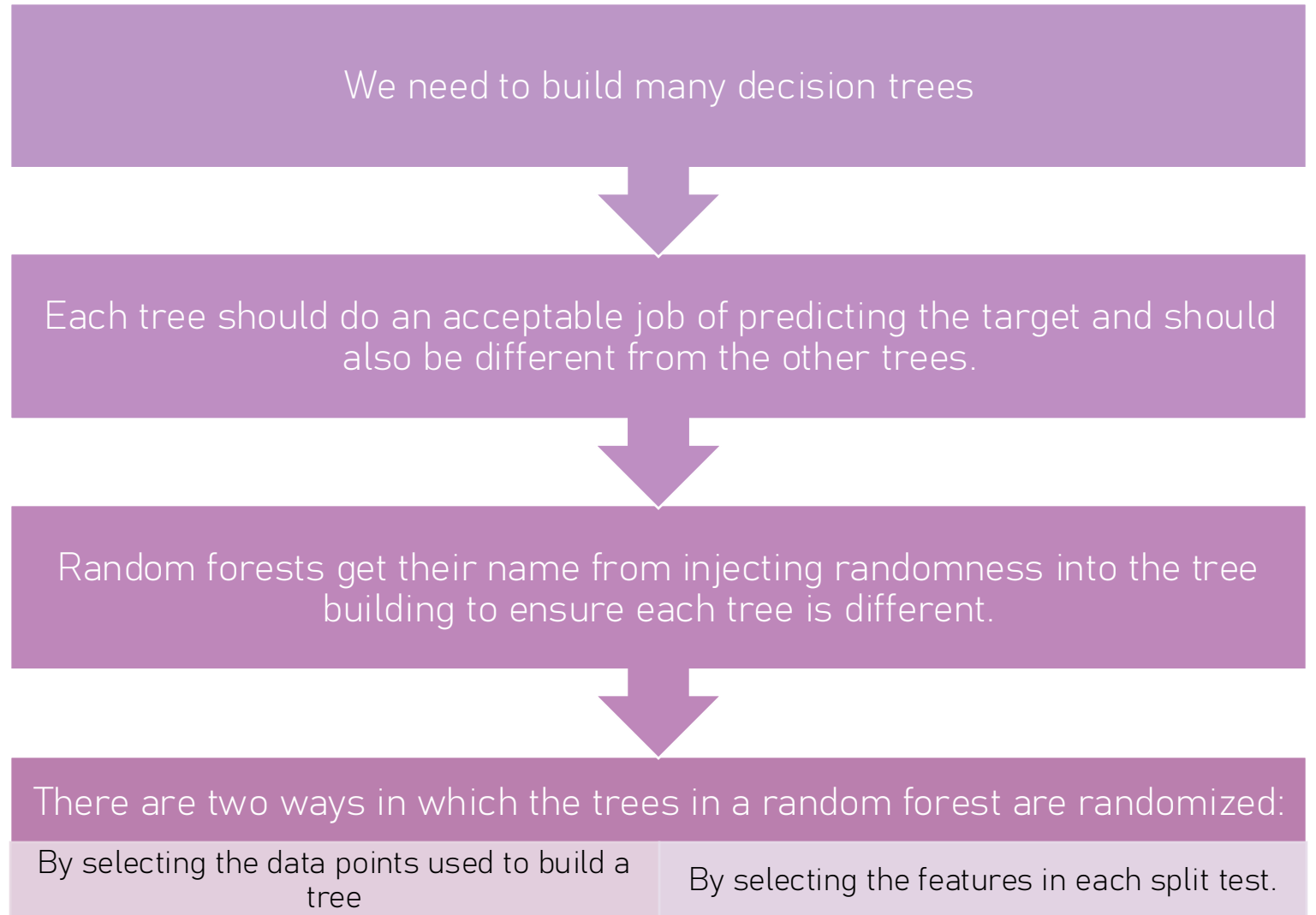
GRADIENT BOOSTED
DECISION TREES.



Random Forests

- A random forest is essentially a collection of decision trees, where each tree is slightly different from the others
- The idea behind random forests is that each tree might do a relatively good job of predicting, but will likely overfit on part of the data.
- If we build many trees, all of which work well and overfit in different ways, we can reduce the amount of overfitting by averaging their results.
- This reduction in overfitting, while retaining the predictive power of the trees, can be shown using rigorous mathematics.

To implement this strategy:



Building Random Forests



1. To build a random forest model, you need to decide on the number of trees to build (the **n_estimators** parameter of RandomForestRegressor or RandomForestClassifier).



Let's say we want to build 10 trees. These trees will be built completely independently from each other, and the algorithm will make different random choices for each tree to make sure the trees are distinct.



To build a tree: we first take what is called a bootstrap sample of our data

That is, from our **n_samples** data points, we repeatedly draw an example randomly with replacement (meaning the same sample can be picked multiple times), **n_samples** times. This will create a dataset that is as big as the original dataset, but some data points will be missing from it (approximately one - third), and some will be repeated.

Building Random Forests



To illustrate, let's say we want to create a bootstrap sample of the list ['a', 'b', 'c', 'd']. A possible bootstrap sample would be ['b', 'd', 'd', 'c']. Another possible sample would be ['d', 'a', 'd', 'a'].



Next, a decision tree is built based on this newly created dataset. However, the algorithm we described for the decision tree is slightly modified.



Instead of looking for the best test for each node, in each node the algorithm randomly selects a subset of the features, and it looks for the best possible test involving one of these features.



The number of features that are selected is controlled by the `max_features` parameter.

Building Random Forests



A critical parameter in this process is `max_features`. If we set `max_features` to `n_features`, that means that each split can look at all features in the dataset, and no randomness will be injected in the feature selection (the randomness due to the bootstrapping remains, though).



If we set `max_features` to 1, that means that the splits have no choice at all on which feature to test and can only search over different thresholds for the feature that was selected randomly.



Therefore, a high `max_features` means that the trees in the random forest will be quite similar, and they will be able to fit the data easily, using the most distinctive features.



A low `max_features` means that the trees in the random forest will be quite different and that each tree might need to be very deep in order to fit the data well.

Random Forests Prediction for Regression & Classification

Regression:

- To make a prediction using the random forest, the algorithm first makes a prediction for every tree in the forest. For regression, we can average these results to get our final prediction.

Classification:

- A “soft voting” strategy is used. This means each algorithm makes a “soft” prediction, providing a probability for each possible output label. The probabilities predicted by all the trees are averaged, and the class with the highest probability is predicted.

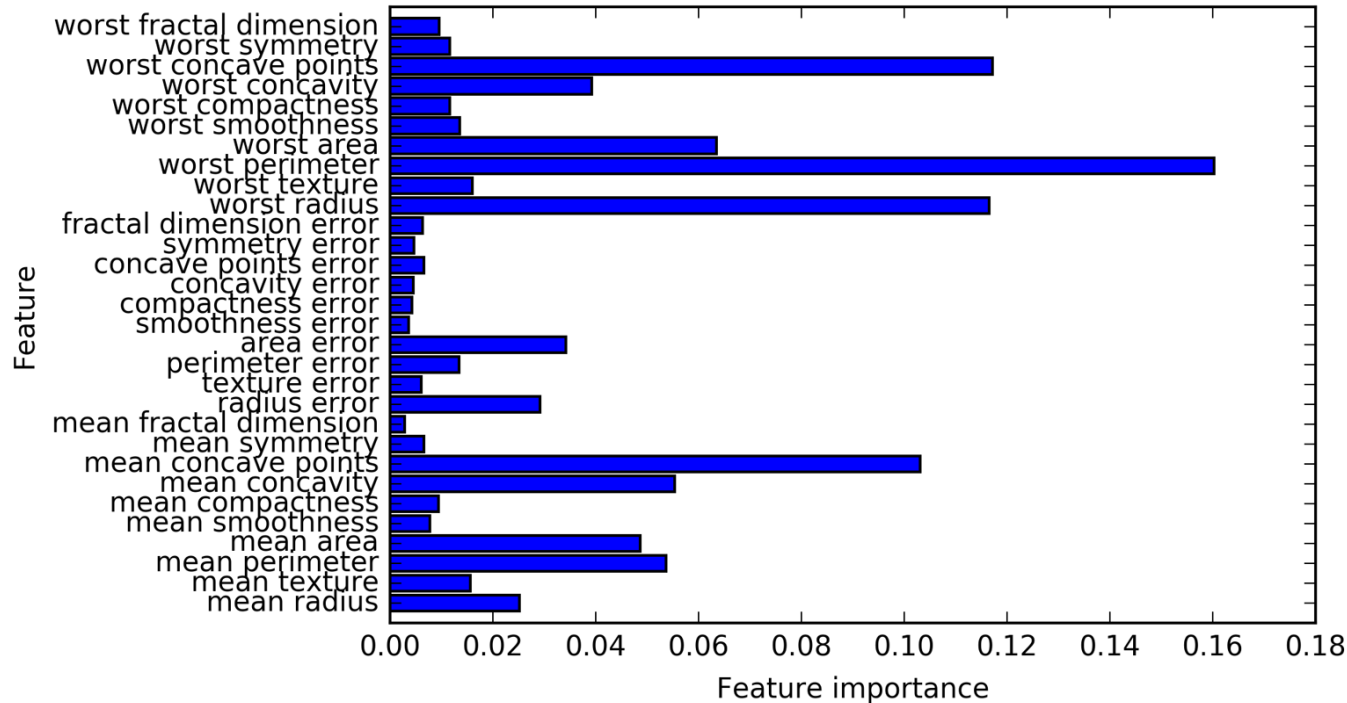
Let's apply a random forest consisting of five trees to the two_moons dataset we studied earlier.

- `from sklearn.ensemble import RandomForestClassifier`
- `from sklearn.datasets import make_moons`
- `X, y = make_moons(n_samples=100, noise=0.25, random_state=3)`
- `X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,`
- `random_state=42)`
- `forest = RandomForestClassifier(n_estimators=5, random_state=2)`
- `forest.fit(X_train, y_train)`
- Let's analyze this from the provided ipynb file:

Applying Random Forest with 100 trees on the Breast Cancer dataset:

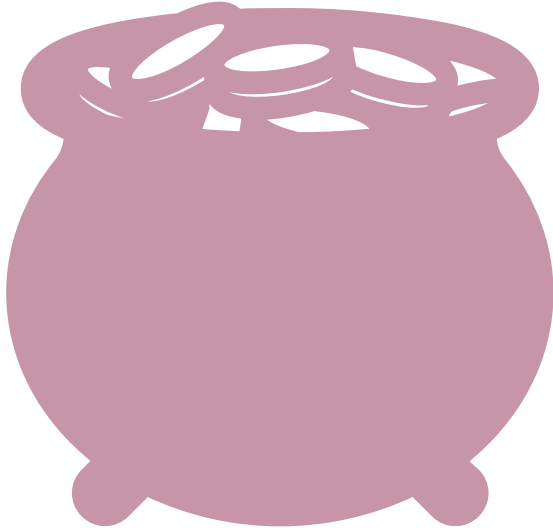
- `X_train, X_test, y_train, y_test = train_test_split(`
- `cancer.data, cancer.target, random_state=0)`
- `forest = RandomForestClassifier(n_estimators=100, random_state=0)`
- `forest.fit(X_train, y_train)`
- `print("Accuracy on training set: {:.3f}".format(forest.score(X_train, y_train)))`
- `print("Accuracy on test set: {:.3f}".format(forest.score(X_test, y_test)))`

Analyses



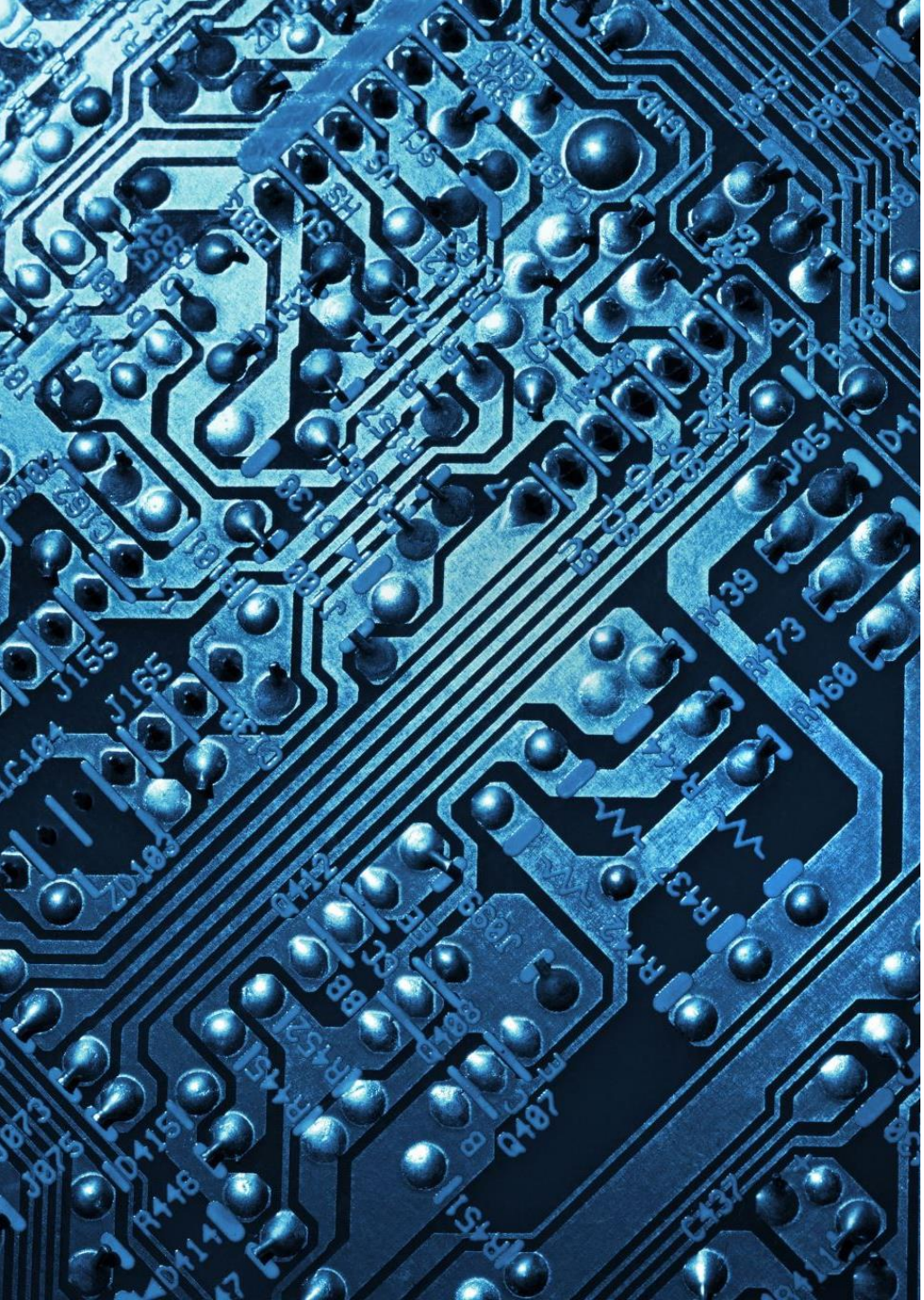
Feature importances computed from a random forest that was fit to the Breast Cancer dataset

- The random forest gives us an accuracy of 97%, better than the linear models or a single decision tree, without tuning any parameters. We could adjust the `max_features` setting, or apply pre-pruning as we did for the single decision tree. However, often the default parameters of the random forest already work quite well.
- Similarly to the decision tree, the random forest provides feature importances, which are computed by aggregating the feature importances over the trees in the forest. Typically, the feature importances provided by the random forest are more reliable than the ones provided by a single tree.



Strengths, weaknesses, and parameters of Random Forest

- Random forests for regression and classification are currently among the most widely used machine learning methods.
- They are very powerful, often work well without heavy tuning of the parameters, and don't require scaling of the data.
- Random forests share all of the benefits of decision trees while making up for some of their deficiencies.
- One reason to still use **decision trees** is if you need a compact representation of the decision-making process. Otherwise, use random forest.
- Building random forests on large datasets can be time consuming.



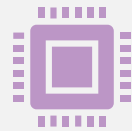
Solution to Time Consumption When Building Random Forests on Large Datasets

- Parallelized across multiple CPU cores within a computer
- This can be easily achieved. How?
- If you are using a multi-core processor (as nearly all modern computers do), you can use the **n_jobs** parameter to adjust the number of cores to use.
- Using more CPU cores will result in linear speed-ups (using two cores, the training of the random forest will be twice as fast), but specifying **n_jobs** larger than the number of cores will not help
- You can **set n_jobs=-1** to use all the cores on your computer.
- The more trees there are in the forest, the more robust it will be against the choice of random state. If you want to have reproducible results, it is important to fix the `random_state`.

Weakness of Random Forest



Random Forest doesn't perform well on very high dimensional, sparse data, such as text data



Random forests usually work well even on very large datasets, and training can easily be parallelized over many CPU cores within a powerful computer. However, random forests require **more memory and are slower to train and to predict** than linear models.

Before we move on, note the following about Random Forest

The important parameters to adjust are `n_estimators`, `max_features`, and possibly pre-pruning options like `max_depth`. For `n_estimators`, larger is always better.

Averaging more trees will yield a more robust ensemble by reducing overfitting. However, there are diminishing returns, and more trees need more memory and more time to train.

A common rule of thumb is to build “as many as you have time/memory for”.

As described earlier, `max_features` determines how random each tree is, and a smaller `max_features` reduces overfitting. In general, it's a good rule of thumb to use the default values: `max_features=sqrt(n_features)` for classification and `max_features=log2(n_features)` for regression.

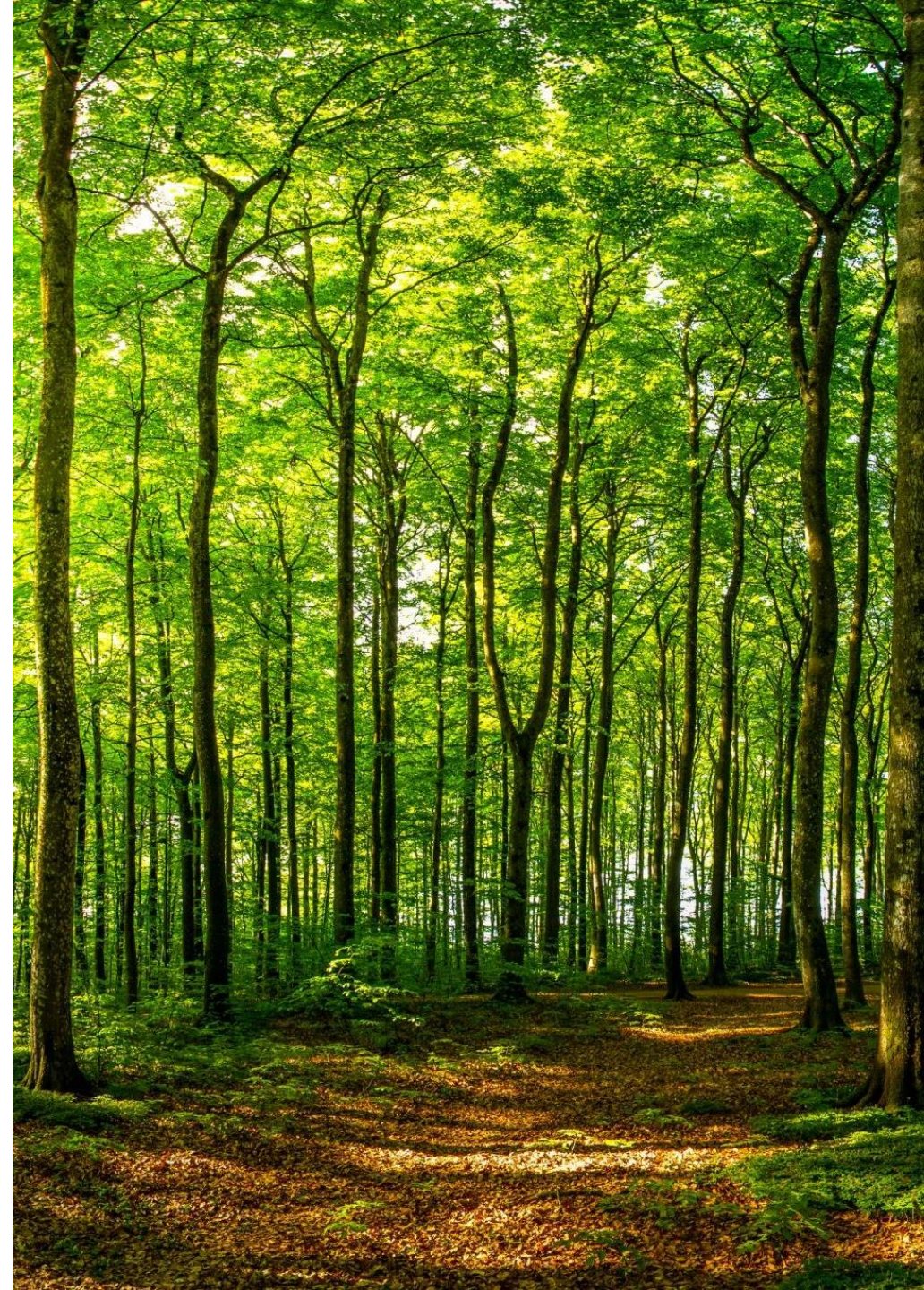
Adding `max_features` or `max_leaf_nodes` might sometimes improve performance. It can also drastically reduce space and time requirements for training and prediction.

Now that we are done with Random Forest

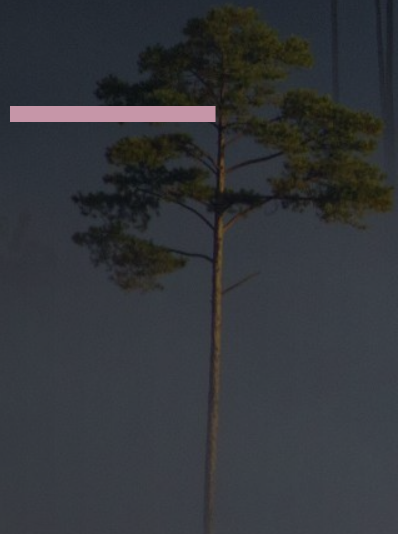
- Let's look at another powerful decision-tree-based ensemble model called gradient-boosted regression trees (gradient-boosting machines)

© DR. ABDULRAUF GIDADO

WITH EXAMPLES AND FIGURES FROM INTRODUCTION TO MACHINE LEARNING
WITH PYTHON: A GUIDE FOR DATA SCIENTISTS BY ANDREAS C.
MÜLLER, SARAH GUIDO · 2016



Gradient boosted regression trees (gradient boosting machines)



Gradient boosted regression trees (gradient boosting machines)



This is another ensemble method that combines multiple decision trees to create a more powerful model.



These models can be used for regression and classification.



In contrast to the random forest approach, gradient boosting works by **building trees in a serial manner, where each tree tries to correct the mistakes of the previous one.**



This implies that by default, there is no randomization in gradient boosted regression trees (Do not confuse the regression in its name to imply that it is only used for regression problem, it can be used for both regression and classification); **instead, strong pre-pruning is used.**



Gradient boosted trees often use very shallow trees, of depth one to five, which makes the model smaller in terms of memory and makes predictions faster.



gradient boosting machines

- The main idea behind gradient boosting is to combine many simple models (in this context known as weak learners), like shallow trees. Each tree can only provide good predictions on part of the data, and so more and more trees are added to iteratively improve performance.



Hurray!

- Gradient boosted trees are frequently the winning entries in machine learning competitions, and are widely used in industry. They are generally a bit more sensitive to parameter settings than random forests, but can provide better accuracy if the parameters are set correctly.

Notes on Gradient boosted regression trees (gradient boosting machines) Complexity



Apart from the pre-pruning and the number of trees in the ensemble, another important parameter of gradient boosting is the **learning_rate**, which controls how strongly each tree tries to correct the mistakes of the previous trees



A higher learning rate means each tree can make stronger corrections, allowing for more complex models.



Adding more trees to the ensemble, which can be accomplished by increasing **n_estimators**, also increases the model complexity, as the model has more chances to correct mistakes on the training set.

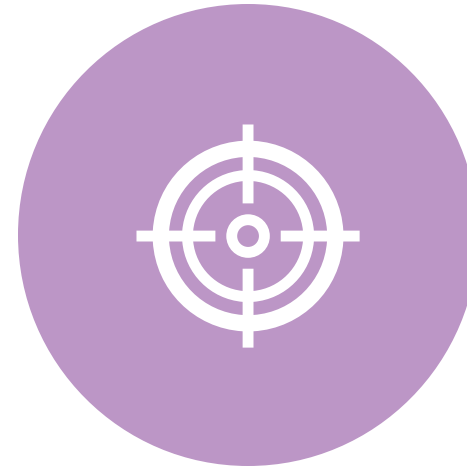
Let's now look at an example of using GradientBoostingClassifier on the Breast Cancer dataset. By default, 100 trees of maximum depth 3 and a learning rate of 0.1 are used:

- `from sklearn.ensemble import GradientBoostingClassifier`
- `X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target, random_state=0)`
- `gbrt = GradientBoostingClassifier(random_state=0)`
- `gbrt.fit(X_train, y_train)`
- `print("Accuracy on training set: {:.3f}".format(gbrt.score(X_train, y_train)))`
- `print("Accuracy on test set: {:.3f}".format(gbrt.score(X_test, y_test)))`

Oh! What do you think from this result?

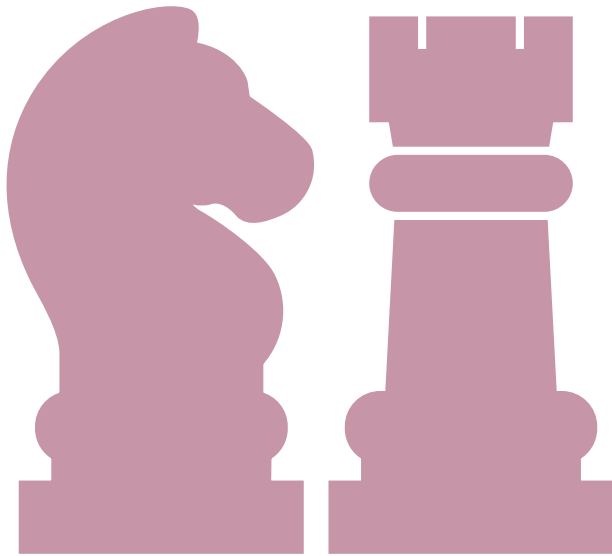


*ACCURACY ON
TRAINING SET: 1.000*



*ACCURACY ON TEST
SET: 0.958*

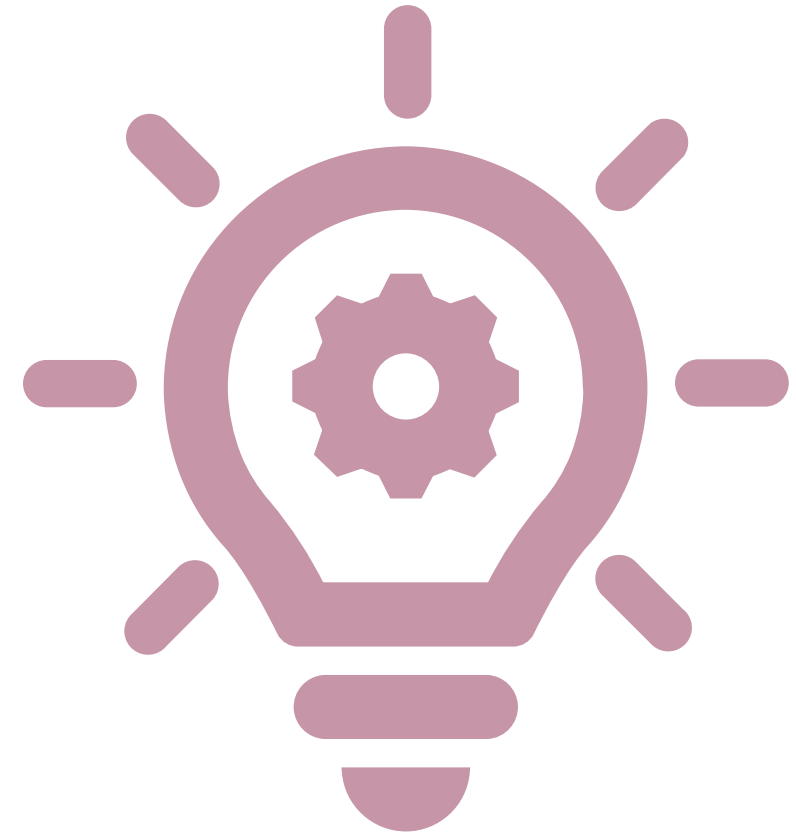
Yes, your assertion is correct!



- As the training set accuracy is 100%, we are likely to be overfitting.

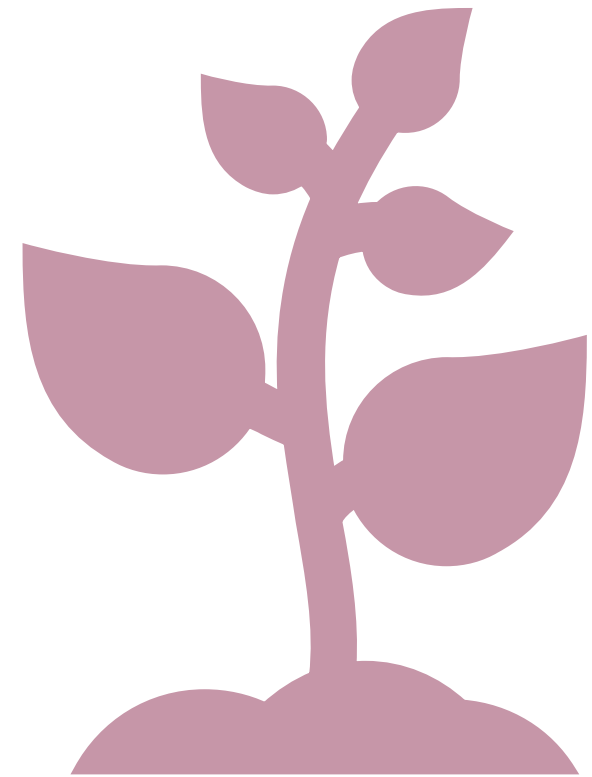
Then, what can we do?

- To reduce overfitting, we could either apply stronger pre-pruning by limiting the maximum depth or lower the learning rate:



Applying stronger pre-pruning by limiting the maximum depth

- `gbrt = GradientBoostingClassifier(random_state=0,
max_depth=1)`
- `gbrt.fit(X_train, y_train)`
- `print("Accuracy on training set:
{:.3f}".format(gbrt.score(X_train, y_train)))`
- `print("Accuracy on test set:
{:.3f}".format(gbrt.score(X_test, y_test)))`
- Output:
- Accuracy on training set: 0.991
- Accuracy on test set: 0.972

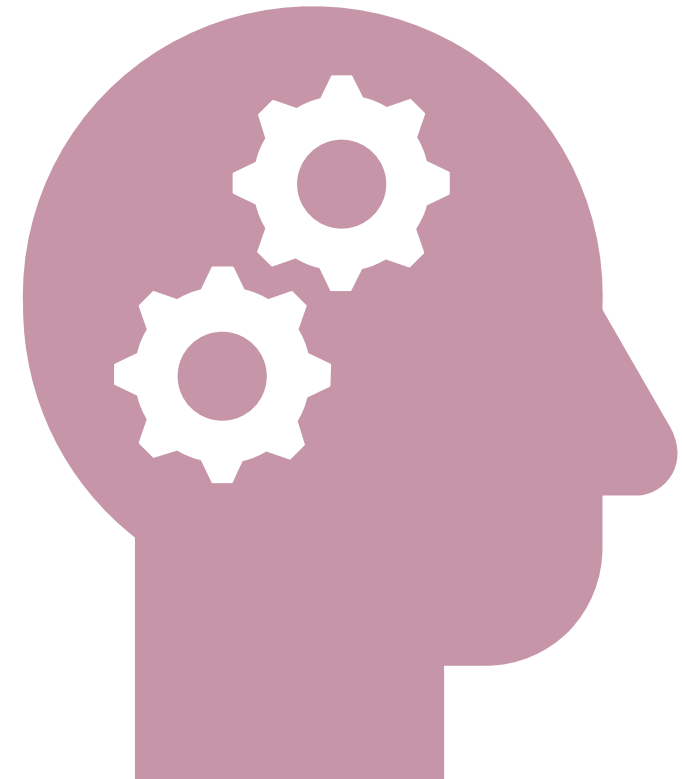


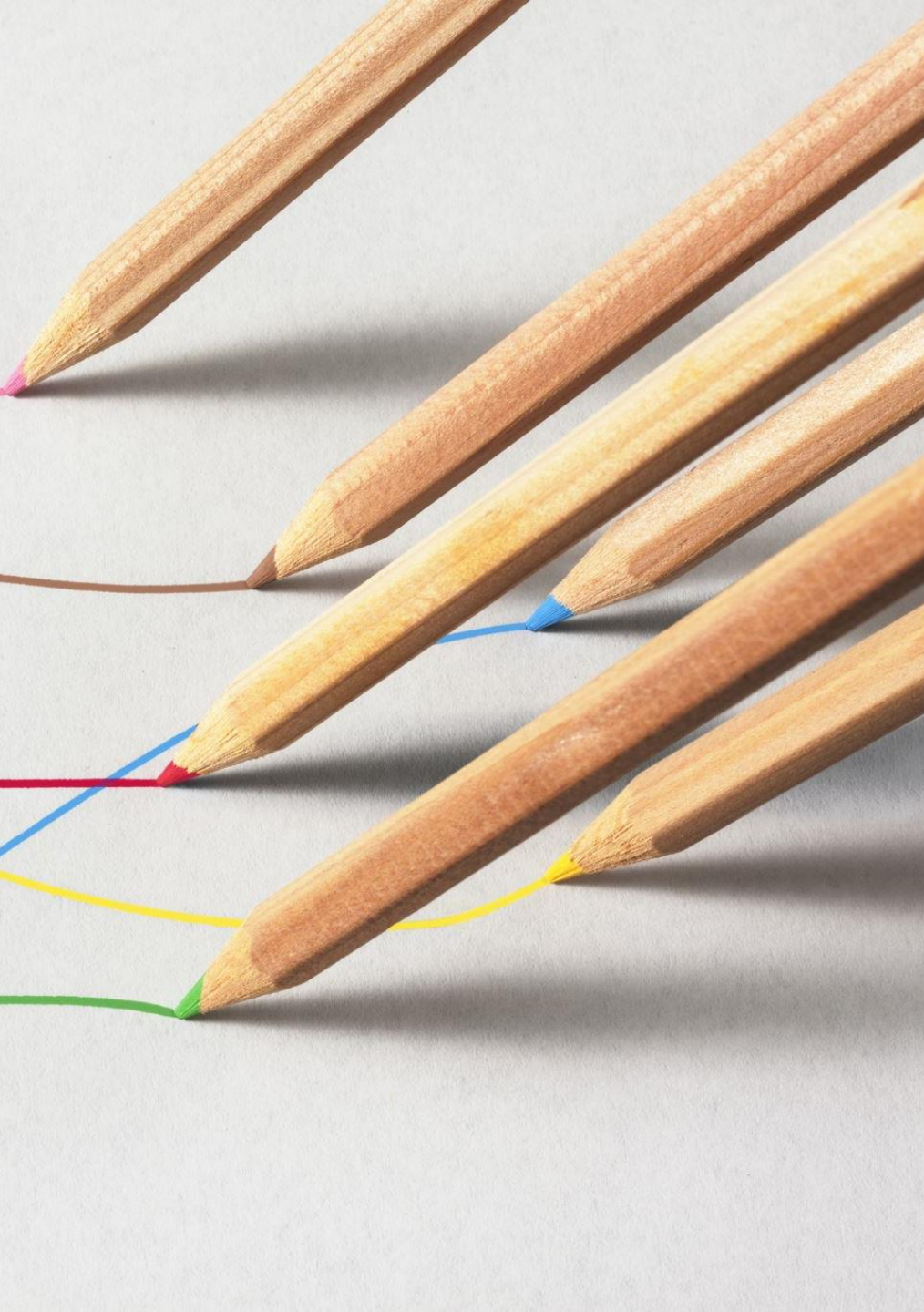
Reducing overfitting by lowering the learning rate

- `gbrt = GradientBoostingClassifier(random_state=0, learning_rate=0.01)`
- `gbrt.fit(X_train, y_train)`
- `print("Accuracy on training set: {:.3f}".format(gbrt.score(X_train, y_train)))`
- `print("Accuracy on test set: {:.3f}".format(gbrt.score(X_test, y_test)))`

Output

- Accuracy on training set: 0.988
- Accuracy on test set: 0.965





A short analysis of what we've just done

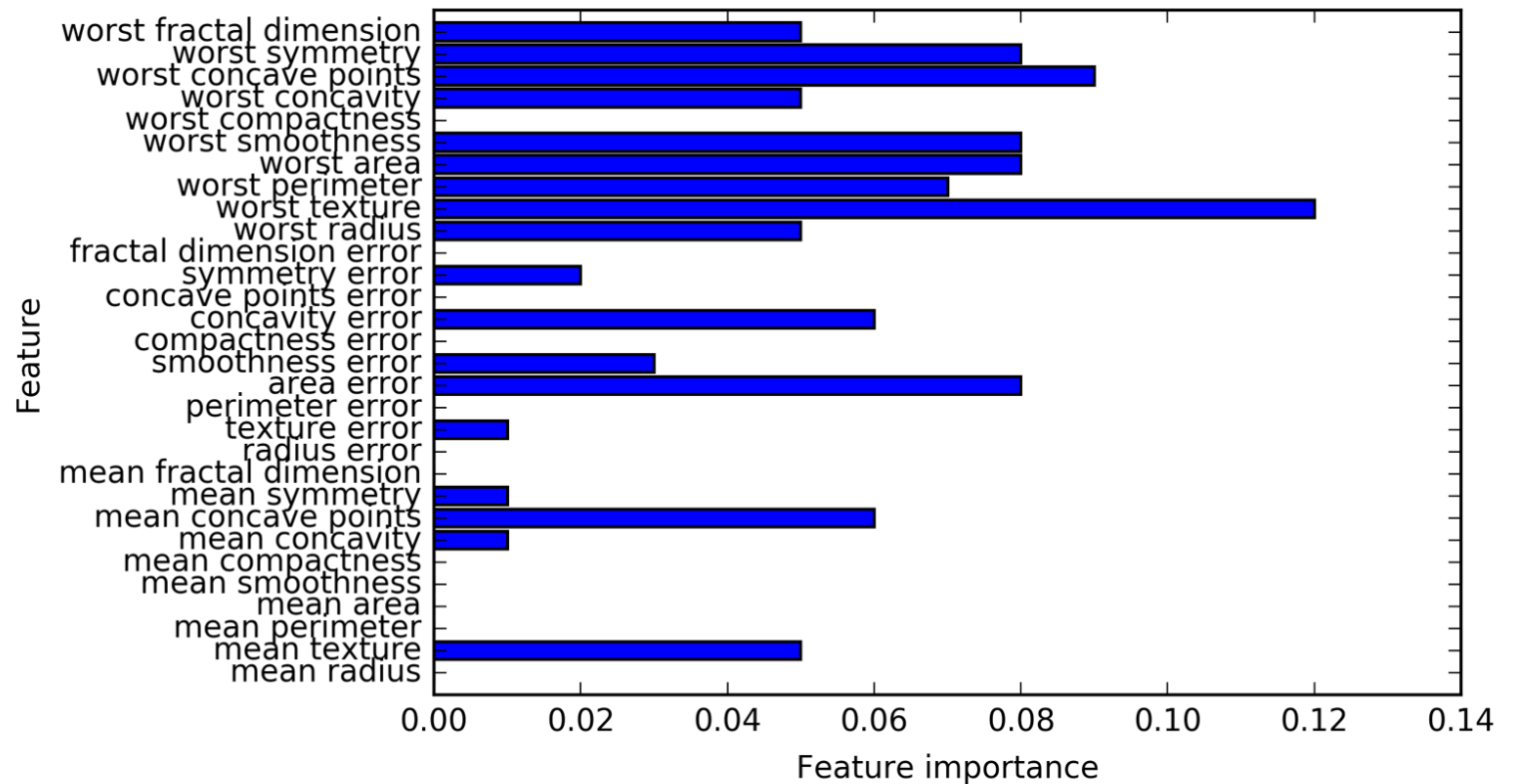
- Both methods of decreasing the model complexity reduced the training set accuracy, as expected.
- In this case, lowering the maximum depth of the trees provided a significant improvement of the model, while lowering the learning rate only increased the generalization performance slightly.

Visualization

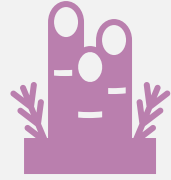


- Just like the other decision tree-based models, we can again visualize the feature's importance to get more insight into our model (Check the previous week's lessons in this slide). As we used 100 trees, it is impractical to inspect them all, even if they are all of depth 1
- `gbrt = GradientBoostingClassifier(random_state=0, max_depth=1)`
- `gbrt.fit(X_train, y_train)`
- `plot_feature_importances_cancer(gbrt)`

Feature importances
computed from a
gradient boosting
classifier that was fit
to the Breast Cancer
dataset



From the features' importance chat:



We can see that the feature importances of the gradient boosted trees are somewhat similar to the feature importances of the random forests, though the gradient boosting completely ignored some of the features.



As both gradient boosting and random forests perform well on similar kinds of data, a common approach is to first try random forests, which work quite robustly. If random forests work well but prediction time is at a premium, or it is important to squeeze out the last percentage of accuracy from the machine learning model, moving to gradient boosting often helps.



If you want to apply gradient boosting to a large-scale problem, it might be worth looking into the **xgboost** package and its Python interface, which is faster (and sometimes easier to tune) than the scikit-learn implementation of gradient boosting on many datasets.

Assignment 3

XGBoost

Kernelized
Support Vector
Machines



Strengths, weaknesses, and parameters gradient boosting



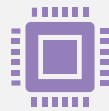
Strengths and weaknesses



Gradient boosted decision trees are among the most powerful and widely used models for supervised learning.



Their main drawback is that they require careful tuning of the parameters and may take a long time to train.



Just like other tree-based models, the algorithm works well without scaling and on a mixture of binary and continuous features.



As with other tree-based models, it also often does not work well on high-dimensional sparse data.

Parameters: gradient boosting



The main parameters of gradient boosted tree models are the number of trees, **n_estimators**, and the **learning_rate**, which controls the degree to which each tree is allowed to correct the mistakes of the previous trees.



n_estimators, and the **learning_rate** are highly interconnected



lower **learning_rate** means that more trees are needed to build a model of similar complexity



In contrast to random forests, where a higher **n_estimators** value is always better, **increasing n_estimators in gradient boosting leads to a more complex model, which may lead to overfitting.**



A common practice is to fit **n_estimators** depending on the time and memory budget, and then search over different **learning_rates**.



Another important parameter is **max_depth** (or alternatively **max_leaf_nodes**), to reduce the complexity of each tree. Usually **max_depth** is set very low for gradient boosted models, often not deeper than five splits.